ASSUMPTION UNIVERSITY
Vincent Mary School of Engineering, Science and Technology
CSX3007 QUIZII (70 points) Semester 2/2025 **SOLUTION**

Instructions: A non-programmable scientific calculator is allowed. It is a **closed-book** exam. Write your **ID** and **Name** on the answer sheet. An answer sheet with a pencil is not accepted.

1. (6 points) Assume that a **3.5GHz** unpipelined processor executes a benchmark program with the following instruction mix and clock cycle count shown in the following Table:

Instruction	Instruction Count	Clock cycle count
Integer arithmetic	4500	3
Data transfer	3500	2
Floating point	2400	4
Control transfer	1500	5

Determine the program's **effective CPI, MIPS rate, and Execution time (CPU time)**.
Effective CPI = 3.16, MIPS rate = 1108, and Execution time (CPU time) = 10.743 x 10^{-6} sec.

2. (1 point) Processor **A** performs a given task in **3** seconds while processor **B** requires **5** seconds for the same task. By what percentage is **A** faster than **B**? **40%**
3. (2 points) Consider **four jobs** being run on a corporate computer. The observed job execution rates per hour were **0.5**, **0.45**, **0.53**, and **0.43**. What is the *central tendency* of these measurements in jobs per hour? **4/(1/0.5 +1/0.45 + 1/0.53 + 1/0.43) = 0.476**
4. (4 points) Assume that two computers (**A** and **B**) execute **four loops** of a scientific program. The details of their clock cycles for each loop are shown in the **Table** below. For a particular benchmark, **loop1** is executed **30** times, **loop2** is executed **40** times, **loop3** is executed **50** times, and **loop4** is executed **60** times. What is the central tendency in the speedup of the computers (from A to B)?

Loop	Comp A	Comp B
1	40	30
2	50	20
3	30	25
4	24	15

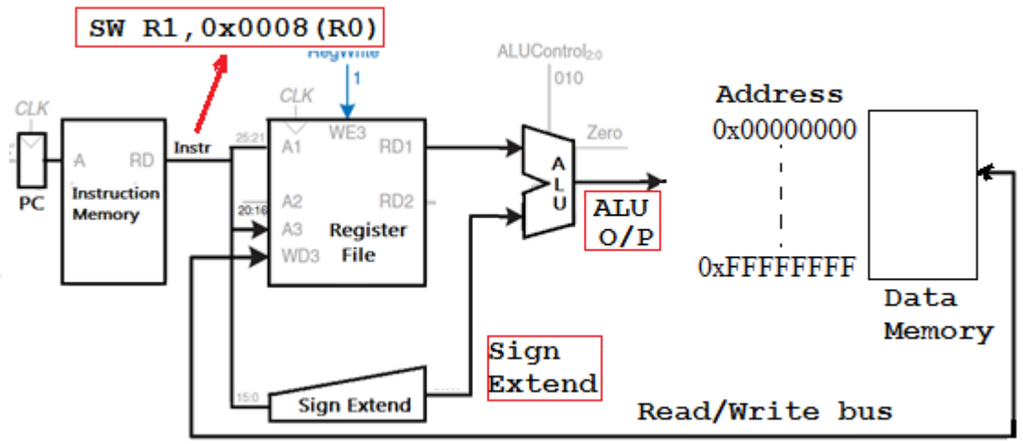
Weighted Geometric mean = 1.5796

Loop	Comp A	Comp B	Speed	Weight
1	**40**	**30**	**1.33**	**0.17**
2	**50**	**20**	**2.5**	**0.22**
3	**30**	**25**	**1.2**	**0.28**
4	**24**	**15**	**1.6**	**0.33**

5. (4 points) A benchmark test on two server systems, P4600 (an 8-core system) and H6600 (a 4-core system), has produced the following results: the P4600 shows 40% overhead during the test, whereas the H6600 has demonstrated the best performance during 70% of its test time. So, which system is more efficient for running your task?
Speedup = 1/((1-*a*) + *a*/*n*)
P4600: *a* = 0.6, *n* = 8, speedup = 2.11 **H6600: *a* = 0.7, *n* = 4, speedup = 2.11.**
Both systems are efficient.
6. (2 points) What are the **four state elements** of the MIPS system? **PC register, IM, RF, and DM**
7. (6 points) Answer the following based on the MIPS instruction format given in the **Figure** below:
   7.1. (1 point) What is the instruction type (in terms of the operands)? **R-type**
   7.2. (1 point) Show an example instruction (in assembly code). **SUB R1, R2, R3**
   7.3. (1 point) How many opcodes are possible? **64**
   7.4. (1 point) How many registers are in the system? **32**
   7.5. (1 point) How many functions are in the system? **64**
   7.6. (1 point) What is the size of the CPU? **32**

op	rs	rt	rd	shamt	funct
6 bits	5 bits	5 bits	5 bits	5 bits	6 bits

8. (7 points) Answer the following questions based on the single-cycle data path of the MIPS system shown in the **Figure** below. Assume that the registers **R0** and **R1** have initialized with **0x00000007** and **0xC123FAE7**:
   8.1. (1 point) Show the **I/P of the Sign Extend unit** (in hexadecimal). **0x0008**
   8.2. (1 point) Show the **O/P of the Sign Extend unit** (in hexadecimal). **0x00000008**
   8.3. (1 point) Show the **O/P of the ALU** (in hexadecimal). **0x0000000F**
   8.4. (4 points) Show the result of the SW operation in Data Memory (each byte with its complete byte address).
   0x0000000F: 0xC1
   0x00000010: 0x23
   0x00000011: 0xFA
   0x00000012 : 0xE7



9. (4 points) Consider the following MIPS assembly code (where R0, R1, and R2 are 32-bit general-purpose registers, and R0 has initialized with a value of **0x00000001**):
   i) `ADDI R0, R0,0x0001`
   ii) `LW R1, 0x0003(R0)`
   iii) `SLL R0, R0, 0x0002`
   iv) `SW R2, 0x0002(R0)`

   Based on the code sequence, answer the following: **(a).** (1point) Show the value (hexadecimal) of the register **R0** after the execution of the instructions (i) and (iii). **0x00000008** **(b).** (3 points) Show the **physical memory addresses** (hexadecimal) of the instructions (ii) **0x00000005** and (iv). **0x0000000A**
10. (2 points) A high-level language compiler running on a MIPS system has the following source code. Show the complete assembly code (where ***g*** and ***h*** are integer variables).

```
if (g > h)
  g = g + h;
else
  g = g - h;
```

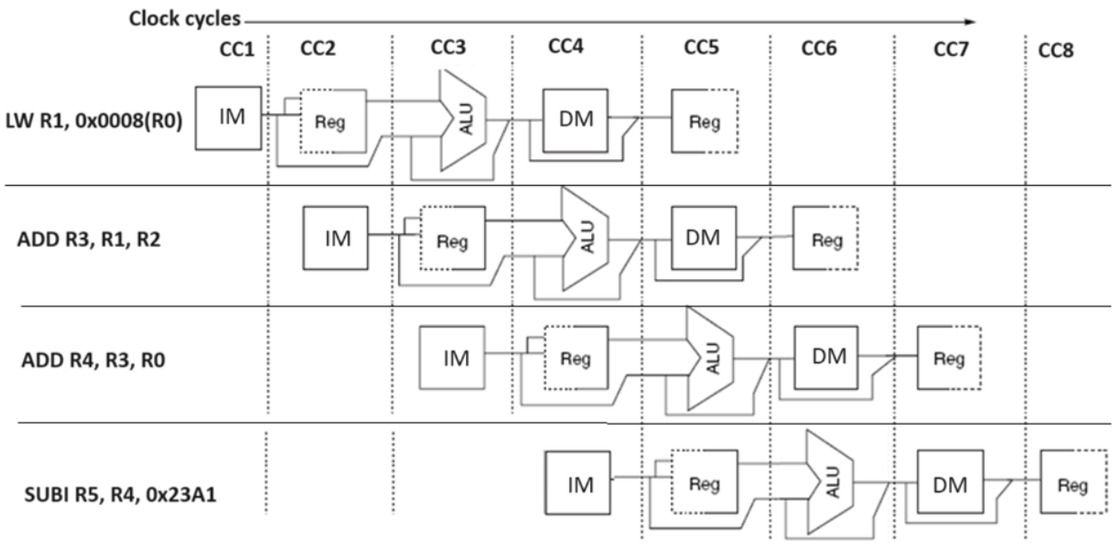
```
LW R1, address g  // address of g is not available
LW R2, address h  // address of h is not available
BNG R1, R2, SUBTRACT  // BNG means if R1 !> R2 ,
ADD R1, R1, R2
J EXIT
SUBTRACT: SUB R1, R1, R2
EXIT:
========OR====================
LW R1, address g
LW R2, address h
BG R1, R2, ADDITION  // BG means R1 > R2 ,
SUB R1, R1, R2
J EXIT
ADDITION: ADD R1, R1, R2
EXIT:
```

11. (3 points) Based on the given instruction sequence, determine the **values** (in hexadecimal) of the registers **R1**, **R2,** and **R3** after the instruction execution (assume that the register **R0** is initialized with **0x00000001**).

```
1.   ADDI       R1, R0, 0x0005
2.   SLL        R1, R1, 1
3.   ADDI       R2, R0, 0x000A
4.   BEQ        R1, R2, NEXT
5.   SUBI       R1, R0, 8
6.   ADD        R3, R2, R0
7.   NEXT: ADDI R3, R2, 0x0002
```

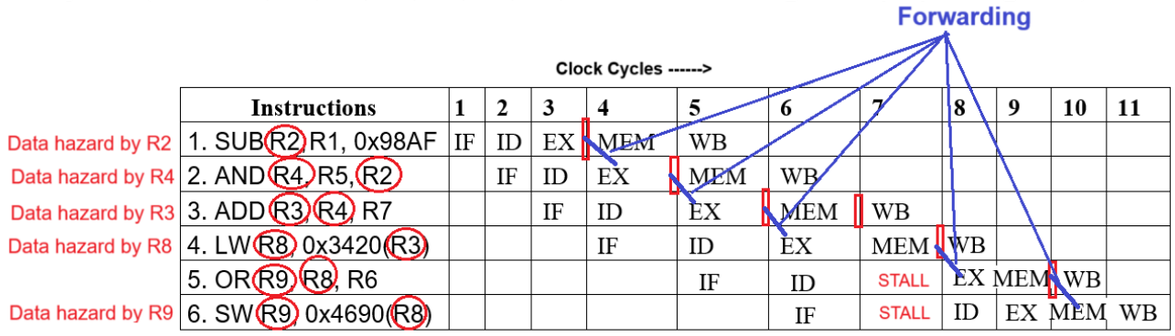
R1 = 0xFFFFFFF9
R2 = 0x0000000B
R3 = 0x0000000D

12. (2 points) What are the pipeline stages in the 32-bit MIPS system?
13. (2 points) Consider an ***S-stage*** pipeline system executing a program with ***K*** instructions. Show that the pipeline speedup equals ***K*** when ***S*** goes to infinity.
    Speedup = SK/(S+K-1), After dividing both numerator and denominator by S: K/(1 + (K-1)/S), if S → ∞, Speedup = K
14. (4 points) Consider an **unpipelined** processor. It has a **2 ns** clock cycle and uses **4 cycles for ALU operations, 5 for branches, and 6 for memory operations**. Assume that the relative frequencies of these operations are **50%**, **15%**, and **35%**, respectively. **Pipelining** the processor adds **0.4 ns** of clock overhead (ignore other pipeline latencies and hazards). Find out how much speedup the instruction execution rate will gain from the pipeline.
    Avg.Inst.Exe.time(un-pipeline) = 9.7ns, Avg.Inst.Exe.time(pipeline) = 2.4ns.
    Speedup = 9.7ns/2,4ns = 4.042 times
15. (1 point) Assume that the MIPS pipelined system has imposed a stall of **0.78** clock cycles per instruction for a particular user process. What is the **CPI** of the system? **1.78**
16. (3 points) What is a *structural hazard*? Consider the **pipeline data path** of the MIPS system shown in the **Figure** below. Is there any structural hazard in the system? If yes, describe the possible ways (if any) to eliminate it. **Resource conflicts in pipeline execution are called structural hazards. There is no structural hazard with the pipeline operation given.**



17. (4 points) Consider the given MIPS instruction sequence:
    1. SUBI R2, R1, 0x98AF
    2. AND R4, R5, R2
    3. ADD R3, R4, R7
    4. LW R8, 0x3420(R3)
    5. OR R9, R8, R6
    6. SW R9, 0x4690(R8)

Using the MIPS **pipeline sequence diagram**, identify the **Read After Write (RAW)** data hazards among the instructions (if any) by indicating the **hazard-causing registers with their instruction numbers**. If any RAW data hazard is detected in the instructions, show the solution steps taken by the MIPS system (where R1, R2, R3, R4, R5, R6, R7, and R8 are 32-bit general-purpose registers).



	Instructions	1	2	3	4	5	6	7	8	9	10	11
Data hazard by R2	1. SUB R2, R1, 0x98AF	IF	ID	EX	MEM	WB						
Data hazard by R4	2. AND R4, R5, R2		IF	ID	EX	MEM	WB					
Data hazard by R3	3. ADD R3, R4, R7			IF	ID	EX	MEM	WB				
Data hazard by R8	4. LW R8, 0x3420(R3)				IF	ID	EX	MEM	WB			
	5. OR R9, R8, R6					IF	ID	STALL	EX	MEM	WB	
Data hazard by R9	6. SW R9, 0x4690(R8)						IF	STALL	ID	EX	MEM	WB

18. (3 points) Below is a section of the MIPS's **instruction memory (IM)**. Based on that:
    18.1. (1 point) What is the **Program Counter** (**PC**) value if the **IF stage** selects the instruction **0x5B4C6732**? **0x0000000C**
    18.2. (1 point) Show the value of the **PC** when the previous instruction was fetched. **0x00000008**
    18.3. (1 point). Suppose the value of the **PC** is **0x0000000D**. Show the fetched instruction. **0x4C6732AF**

Memory Address	Memory Word
0x00000008	0x23
0x00000009	0x5A
0x0000000A	0x1E
0x0000000B	0xA0
0x0000000C	0x5B
0x0000000D	0x4C
0x0000000E	0x67
0x0000000F	0x32
0x00000010	0xAF

19. (2 points) Why does the MIPS system use a **5-bit address** to access its register file? **There are 32 registers in the register file.**
20. (8 points) Consider the following **high-level language code** running on a 32-bit MIPS system. **Show its MIPS assembly code translation, including complete memory operations with the data memory (DM)** for the variable allocation. The size of the integer value in each variable is **32-bit**. The **32-bit memory address** of the first byte of the variable***, a,*** is **0xA500C004.** The variables ***b*** and ***c*** use the consecutive byte addresses for their values in **DM**. Use the 32-bit registers **R1**, **R2**, and **R3** for ***a,*** ***b***, and ***c*** in memory and arithmetic operations, and register **R0 as the base register**, initialized to **0xA5000002**.

```
int a = 30;
int b = 15 - a;
int c = b + 10;
```

R1 ← a, R2 ← b, R3 ← c (given in the question)

a = 30 = 0x0000001E (the variable a is initialized with 30)

Base register for LW and SW operations is R0 = 0xA5000002 (given in the question)

Starting address of the first byte of 'a' in the DM = 0xA500C004 (given in question)

MIPS instructions for the high-level code:

1. **LW R1, 0xC002(R0) ; R1 = 0x0000001E (30)**
2. **ADDI R2, R2, 0x000F ; R2 = 0x0000000F (15)**
3. **SUB R2, R2, R1 ; R2 = 0x000000F1 (-15)**
4. **SW R2, 0xC006(R0)**
5. **ADDI R3, R2, 0x000A ; R3 = 0xFFFFFFFB (-5)**
6. **SW R3, 0xC00A(R0)**

The DM is utilized for variable storage:

32-bit Address	Data Byte	
0xA500C004	0x00	int a (32-bit) = 0x0000001E (30)
0xA500C005	0x00	
0xA500C006	0x00	
0xA500C007	0x1E	
0xA500C008	0xFF	int b (32-bit) = 0xFFFFFFF1 (-15)
0xA500C009	0xFF	
0xA500C00A	0xFF	
0xA500C00B	0xF1	
0xA500C00C	0xFF	int c (32-bit) = 0xFFFFFFFB (-5)
0xA500C00D	0xFF	
0xA500C00E	0xFF	
0xA500C00F	0xFB	

========== **End** ==========